

Reviewer Pack — Minimal Rank of Local Systems and Resolution Proof Width Cor...

Entry ecf35186018a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 10:45:29 UTC

Minimal Rank of Local Systems and Resolution Proof Width Correlation

Entry ID: ecf35186018a **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 10:45:29 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Local Systems) **Field B** (complexity object): Resolution Proof Complexity

Statement:

For every CNF formula φ with n variables, the minimal rank (mrnk) of a local system associated with the term graph of φ is linearly correlated with its resolution proof width $w(\varphi)$, such that $\text{mrnk} = \Theta(w(\varphi))$. Equivalently: if $|w(\varphi)| \leq k$ for some constant k , then $\text{mrnk}(\varphi) = O(k)$.

Rationale (proposer's reasoning):

Local systems provide a way to study the geometry of varieties over algebraic curves. By associating them with the term graph of CNF formulas, we may uncover a geometric structure that correlates with the complexity of resolution proofs. This connection could expose new insights into both resolution complexity and algebraic geometry.

Taxonomy category: Alexander-Orlik-Solomon Theory × Resolution Proof Complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ff114d8b5d0c8fc2

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if for all CNF formulas φ with n variables ($n \leq 40$), the ratio $|\text{mrnk}(\varphi)|/|w(\varphi)|$ is within a factor of 2 over 30 seeds, where $\text{mrnk}(\varphi)$ is the minimal rank of the local system and $w(\varphi)$ is the resolution proof width.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): -mrank local system AND resolution proof complexity-algebraic geometry local systems AND resolution proof width- $\Theta(w(\phi))$ correlation with minimal rank of local systems

Top relevant hits considered: - [http://arxiv.org/abs/1205.0723v1] Local structure in Mg1-xAlxB2 system by high resolution neutron diffraction - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise) - [http://arxiv.org/abs/1606.05050v1] Proof Complexity Lower Bounds from Algebraic Circuit Complexity - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics - [http://arxiv.org/abs/1409.1534v1] Algorithms in Real Algebraic Geometry: A Survey - [http://arxiv.org/abs/2501.04451v2] Observation of the W -annihilation process $D_s^+ \rightarrow \omega \rho^+$ and measurement of $D_s^+ \rightarrow \varphi \rho^+$ in $D^+ \rightarrow \pi^+ \pi^+ \pi^-$ - [http://arxiv.org/abs/2309.02774v4] First Measurement of the Decay Asymmetry in the pure W-boson-exchange Decay $\Lambda_c^+ \rightarrow \Xi^0 K^+$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2 * n):
            clause = [random.randint(-n, -1), random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def term_graph(cnf):
        graph = {}
        for clause in cnf:
            for literal in clause:
                if literal not in graph:
                    graph[literal] = set()
                for other_literal in clause:
                    if other_literal != literal and -literal not in clause:
                        graph[literal].add(other_literal)
                        graph[other_literal].add(literal)
        return graph
```

```

def minimal_rank(graph):
    n = len(graph)
    rank = 0
    visited = [False] * (2 * n + 1)

    def dfs(node, color):
        if visited[node]:
            return True
        visited[node] = True
        for neighbor in graph[node]:
            if neighbor != -node and not dfs(neighbor, 3 - color):
                return False
        return True

    for i in range(1, n + 1):
        if not visited[i] and not dfs(i, 1):
            rank += 1
    return rank

def resolution_width(cnf):
    clauses = set()
    for clause in cnf:
        clauses.add(tuple(sorted(clause)))

    queue = list(clauses)
    while queue:
        new_clauses = []
        for clause1 in queue:
            for clause2 in queue:
                if len(set(clause1) & set(clause2)) == 1:
                    new_clause = tuple(sorted(list(set(clause1) ^ set(clause2))))
                    if new_clause not in clauses and new_clause not in new_clauses:
                        new_clauses.append(new_clause)
        queue.extend(new_clauses)

    return len(queue)

n = random.randint(5, 40)
cnf = generate_cnf(n)
graph = term_graph(cnf)
mrank = minimal_rank(graph)
w_phi = resolution_width(cnf)

if w_phi == 0:
    return {
        "metric_name": "mrank/w_phi",
        "metric_value": float('inf'),
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "resolution_width_is_zero"
    }

ratio = mrank / w_phi
return {

```

```

        "metric_name": "mrank/w_phi",
        "metric_value": ratio,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": abs(ratio - 2) < 0.5,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.randint(2, 1000003) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        support_fraction = 1.0
        result = f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        result = f"RESULT: FALSIFIED counterexample='mrank/w_phi' first_failing_seed={first_failing_seed}"

    print(result)

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6931774b.py", line 109, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6931774b.py", line 79, in run_trial
    graph = term_graph(cnf)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_6931774b.py", line 37, in term_graph
    graph[other_literal].add(literal)
    ~~~~~
KeyError: 29

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that it does not affect the validity of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	22049
2	preregistration	ollama_remote	glm4:latest	0	0	13182
3	novelty	ollama_remote	glm4:latest	0	0	9017
4	novelty	ollama_remote	glm4:latest	0	0	13864
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21858
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17012
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9408
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13413
9	judge	ollama_remote	glm4:latest	0	0	10068

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 129872 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/ecf35186018a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/ecf35186018a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/ecf35186018a.tar.gz (if generated)